

Decision-Trace Distillation for Reducing LLM Decision Rights in Agentic AI Systems

Oliver Zhou

Lily Li

Abstract

LLM-enabled agentic systems increasingly give large language models operational authority over workflow transitions: interpreting goals, selecting evidence, routing subtasks, choosing tools, evaluating intermediate outputs, writing memory, and deciding whether a process should continue, stop, retry, or escalate. This allocation of decision rights is useful because a single stochastic model can supply semantic flexibility across changing tasks, but it also creates governance costs: consequential workflow behavior becomes difficult to reproduce, inspect, audit, and bound. This paper proposes *decision-trace distillation* as a design pattern for reducing unnecessary runtime decision rights of LLM components without discarding the adaptability that makes them valuable. During a learning period, the LLM holds broader authority and the system records its decisions together with task context, candidate options, rationales, human feedback, downstream outcomes, overrides, uncertainty signals, and scope or risk tags. During a running period, local deterministic or semi-deterministic components are trained on accepted or outcome-validated traces and receive bounded decision rights only when confidence, scope, and risk conditions are satisfied. These components may retrieve precedents, recommend candidate decisions, approve or reject LLM proposals, decide directly above threshold, or participate in a voting procedure. The contribution is not merely a cost-saving variant of model distillation. It reframes distillation as an authority-shifting mechanism inside agentic systems: repeated decisions become more local, inspectable, and governable, while novel, uncertain, or high-risk cases remain with the LLM or a human reviewer. The paper specifies the mechanism, relates it to routing, memory, knowledge compilation, human-in-the-loop governance, and system-level accountability, and proposes an executable evaluation plan for article-structure selection, cover-image theme selection, and economic-data insight selection.

Introduction

A defining feature of many LLM-enabled agentic systems is that the language model is no longer used only to generate a final text response. It often becomes a runtime controller that interprets the user request, decomposes the task, selects evidence, calls tools, writes or revises prompts, checks intermediate outputs, updates memory, and decides whether the workflow should continue, stop, retry, or escalate. Component-level surveys describe this architecture through modules such as profiling, memory, planning, action, and feedback (L. Wang et al. 2023), while newer work frames autonomous LLM agents in terms of perception, reasoning, planning, memory, and execution (Lamo Castrillo et al. 2025). The design is powerful because it lets a single stochastic model mediate between open-ended human goals and structured computational action.

This flexibility can be described more precisely as an allocation of *decision rights*. Following Zhou and Li (2026), this paper uses the term *decision rights* to mean operational authority over consequential process functions: who selects evidence, who reasons over context, who chooses the next action, who validates

an output, who writes to memory, and who determines whether a workflow state should change. The term does not imply moral or legal rights for AI systems. It is a system-design term for locating effective authority among human actors, deterministic components, local learned components, and stochastic LLM components.

The decision-rights lens matters because agentic AI design is not only a question of what modules are present. Two systems can both contain retrieval, memory, tool use, and human review, yet differ sharply in who actually decides. In one system, the LLM may generate the retrieval query, choose the source, summarize the evidence, select the answer, and write the result to memory. In another, deterministic retrieval, schemas, validators, curated memory, local classifiers, and human approval may constrain those same functions. Typological and taxonomy work usefully classifies agentic systems by observable capabilities, autonomy, adaptability, contextualization, and orchestration (Wissuchek and Zschech 2025; Sapkota, Roumeliotis, and Karkee 2025), but a deployed organization also needs to know how much authority the LLM receives at each decision point.

Giving broad decision rights to an LLM is often rational early in system development. Developers may not yet know the right categories, features, thresholds, or exception paths. The LLM can explore the decision space, translate ambiguous user goals into provisional workflow choices, and produce examples from which a more structured representation can later be built. Cognitive design-pattern work similarly emphasizes observe–decide–act cycles, commitment, reconsideration, memory, and knowledge compilation as recurring elements of agent architectures (Wray, Kirk, and Laird 2025). The problem is that broad LLM decision rights can remain in place after a workflow has become routine. At that point, the system may continue to ask a stochastic general-purpose model to make decisions that have become repetitive, local, and learnable.

This paper proposes *decision-trace distillation* as a mechanism for shifting such recurrent decisions away from the LLM during runtime. During a learning period, the LLM holds broader decision rights and its decisions are logged with context, candidate options, rationale, human feedback, downstream outcomes, overrides, and uncertainty indicators. During a running period, a local classifier, ranker, policy model, retrieval model, or approval gate is trained on these traces. The local component receives a limited decision right only when its prediction is confident, the case falls inside its approved scope, and the risk level permits local action. Otherwise, the case is deferred to the LLM, a human reviewer, or a hybrid procedure.

The proposal does not reject LLMs. It changes their role. The LLM remains a teacher, an explorer of novel cases, a fallback decision maker, and a participant in hybrid decision procedures. The local model becomes an inspectable memory-based decision component for familiar cases. The central question is therefore not, “Can the LLM perform this task?” It is: *which component should hold which decision right for this decision point, under which confidence, scope, and risk conditions?*

The paper makes three claims. First, decision traces are useful because they preserve not only what the LLM decided, but also the context, options, feedback, and outcome evidence needed to judge whether that decision should be reused. Second, distillation should be treated as a transfer of bounded authority rather than as a general replacement of the LLM. Third, evaluation should measure decision-share movement, override risk, calibration, deferral, and traceability, not only final task performance.

Background and motivation

LLM decision rights as a source of adaptability

LLMs are useful in agentic systems because they provide a general semantic interface between unstructured human goals and structured computational action. A user may express an ambiguous goal in natural language, while the system must translate that goal into retrieval choices, workflow steps, tool calls, intermediate checks, and output formats. Unlike a conventional rule engine, an LLM can interpret new contexts without requiring a separately engineered decision tree for every situation. This is why LLM agents are frequently designed as flexible controllers rather than as narrow text generators.

This flexibility is especially valuable when task definitions change frequently. In a content-production workflow, the same agent may need to select the structure of a thought-leadership article, choose the tone of a social post, select the theme of a cover image, or frame an economic-data insight. In an operational workflow, the agent may need to route requests among tools, decide whether a case requires human approval, or choose which prompt strategy should be used. Agentic AI typologies capture this breadth through dimensions such as autonomy, adaptability, knowledge scope, contextualization, operation, and normative alignment ([Wissuchek and Zschech 2025](#)).

Broad LLM decision rights can also help a system discover its own future structure. When categories are not yet known, the LLM can make provisional decisions and generate rationales that reveal the implicit features behind those decisions. These early decisions are not ground truth, but they can serve as exploratory supervision. Their value lies in showing which distinctions repeatedly matter: which features the LLM attends to, where humans override it, which outcomes count as acceptable, and which cases remain too novel or risky for automation.

The governance cost of broad LLM decision rights

The cost of broad LLM decision rights is not limited to API fees or latency. The deeper cost is that important workflow transitions become harder to reproduce, inspect, and contest. The same input can produce different decisions across prompts, model versions, decoding settings, provider-side changes, or hidden safety layers. Even when an LLM produces a fluent rationale, that rationale should not automatically be treated as a faithful explanation of the decision process. Recent work warns against crediting LLMs with decision explanation unless the generated explanations are validated against stronger evidential standards ([W. Wang 2026](#)).

Agentic systems amplify this problem because a single early decision can affect later retrieval, tool use, memory writing, action execution, and output evaluation. Failures in autonomous agents can arise from planning, execution, or response generation, and aggregate success rates can hide process-level failure modes ([Lu, Li, and Huo 2025](#)). System-level interpretability research therefore argues that model-level explanations are insufficient for autonomous multi-step workflows; governance needs causal tracing, provenance, and accountability across the whole system ([Zhu et al. 2026](#)). Safety and security work similarly emphasizes that risks can propagate through prompts, tools, memory, retrieval, and orchestration layers rather than remaining confined to a single model call ([Ghosh et al. 2025](#)).

A rights-aware interpretation is that the LLM should not automatically retain authority simply because it can

produce a plausible decision. In many cases, a deterministic validator, a local model, a retrieval system, or a human reviewer should hold part of the relevant right. Human-in-the-loop architectures show one way to decouple approval logic from the main workflow and preserve controlled autonomy at scale (Cheng and Cheng 2026). The present paper extends that idea: once enough local traces exist, some repeated decisions can be shifted not only to humans or rules, but also to learned local components with explicit thresholds and audit trails.

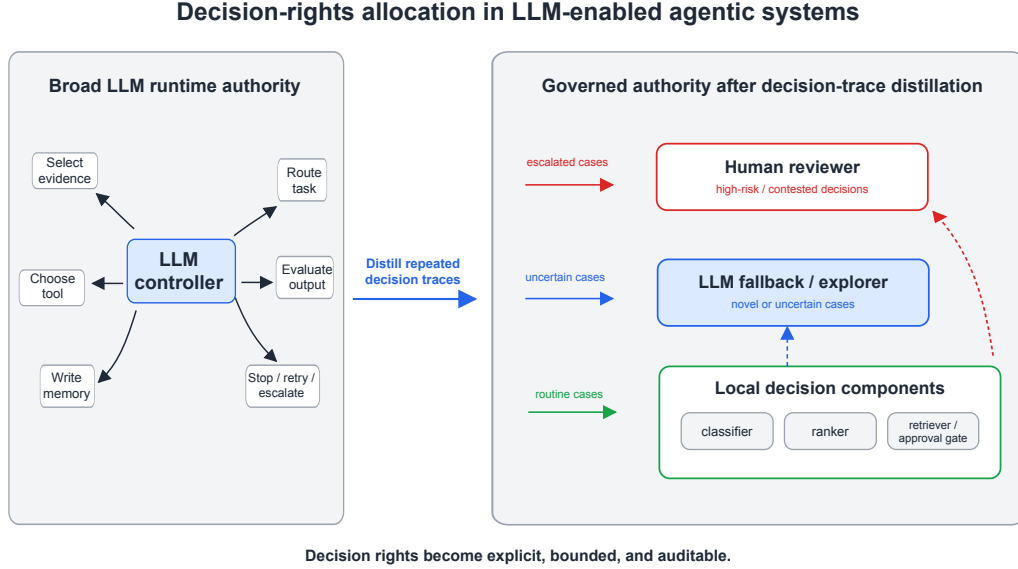


Figure 1: Decision-rights allocation in LLM-enabled agentic systems.

The overview figure (Figure 1) summarizes the shift. The left side represents a common default in which the LLM holds many runtime rights at once: it selects evidence, routes tasks, chooses tools, evaluates output, writes memory, and decides whether to stop, retry, or escalate. The right side represents the proposed design after decision-trace distillation. Routine cases are handled by bounded local components; novel or uncertain cases remain with the LLM; and high-risk or contested decisions are escalated to human review. The point is not that local components are always superior. The point is that decision rights should become explicit, bounded, and auditable.

Relation to routing, distillation, memory, and knowledge compilation

Several existing research streams address adjacent parts of this problem. LLM routing studies how to choose among models with different cost, latency, and quality profiles (Varangot-Reille et al. 2025; Ong et al. 2024; Chen, Zaharia, and Zou 2023). More recent routing work treats model selection as a decision problem that can require causal correction, decision-aligned objectives, or diagnostics for router collapse (Tsiourvas, Sun, and Perakis 2025; Lai and Ye 2026). From the decision-rights perspective, a router holds the right to decide which component acts next. Decision-trace distillation extends this logic beyond model selection: the routed object may be any repeated workflow decision, including evidence selection, memory writing, approval, or escalation.

Model compression and knowledge distillation study how the behavior of large or ensemble models can be

transferred into smaller deployable models (Buciluă, Caruana, and Niculescu-Mizil 2006; Hinton, Vinyals, and Dean 2015). Decision-trace distillation borrows this teacher–student intuition but changes the object being transferred. The goal is not to compress the LLM as a whole. The goal is to transfer a narrow, recurrent *decision right* at a specific workflow point. The student model may be a small classifier, a ranker, a nearest-neighbor retriever, a calibrated approval gate, or a policy model. It does not need to reproduce the LLM’s full reasoning ability.

Memory research also matters. Episodic memory can ground next-task recommendations in workflow precedents (Fiorini et al. 2025), cross-domain agent memory can support reusable problem solving (Tang et al. 2025), and graph-memoized reasoning highlights the need for validation, versioning, and traceability in reusable workflow traces (Singh 2025). Decision-trace distillation treats memory not only as context supplied back to the LLM, but also as training material for a local decision component. This connects the proposal to cognitive ideas of knowledge compilation: repeated reasoning episodes can be converted into faster and more bounded procedural knowledge (Wray, Kirk, and Laird 2025).

The mechanism also connects to domain-expert knowledge codification. Knowledge-protocol and expert-codification approaches argue that reusable expert rules and procedures can specialize a general model without retraining it (Zhang 2025; Uulu et al. 2026). Decision-trace distillation is complementary because it learns from situated decisions and feedback after the system has been operating. Expert codification often starts from explicit human knowledge; decision-trace distillation starts from operational traces and asks which recurrent decisions have become reliable enough to localize.

Proposed mechanism

What is distilled?

The object of distillation is not the LLM’s general linguistic competence. It is a decision trace at a repeated decision point. A decision trace records the relationship among a task context, candidate options, an LLM decision, a rationale or structured explanation, the final accepted decision, and later evidence about whether that decision was useful. In other words, it captures how an initially stochastic decision became accepted, revised, rejected, or escalated inside a workflow.

A minimal trace should include the following fields:

Field	Purpose
Decision point	Identifies the workflow node, such as article-structure selection, source-routing, memory writing, or escalation.
Input context	Stores task description, user goal, source material, metadata, constraints, and relevant history.
Candidate options	Records the finite or semi-finite choice set available at the decision point.

Field	Purpose
LLM decision	Stores the model’s selected option and any structured output.
LLM rationale	Provides a human-readable account, treated as a useful artifact but not as conclusive explanation.
Final accepted decision	Stores the human-accepted, production-accepted, or outcome-validated decision.
Feedback and override	Records approvals, revisions, rejections, escalations, and reasons for intervention.
Outcome signal	Stores downstream metrics such as revision burden, user acceptance, task success, or risk flags.
Scope and risk tags	Marks whether the case is routine, novel, high-risk, regulated, or outside the local model’s allowed domain.
Version and provenance	Records prompt version, model version, tool state, data source, local-model version, and timestamp.

This list is not arbitrary. It combines requirements that appear across several adjacent literatures. Agent evaluation requires trajectory and process records rather than only final-output scores (Yehudai et al. 2025; Zhuge et al. 2024). System-level accountability requires provenance across prompts, tools, memory, and orchestration steps (Zhu et al. 2026). Graph-memoized and episodic-memory approaches require reusable, validated, and versioned traces rather than unstructured recollections (Singh 2025; Fiorini et al. 2025). Human-in-the-loop and safety-oriented work requires explicit override, approval, and risk records (Cheng and Cheng 2026; Ghosh et al. 2025). Finally, because generated rationales are not necessarily faithful decision explanations, the trace should separate the LLM’s stated rationale from accepted decisions and outcome evidence (W. Wang 2026).

The central design choice is to train the local model on accepted or outcome-validated decisions rather than on raw LLM outputs alone. If early LLM decisions are simply copied, the system may crystallize LLM errors. Human feedback, overrides, and downstream outcomes should therefore become part of the supervision signal whenever they are available. The target is not “what the LLM said” but “what the workflow eventually accepted, and under what conditions.”

Learning period and running period

The mechanism has two phases. During the *learning period*, the LLM receives broader decision rights because the workflow has not yet accumulated enough local experience. It observes inputs, selects options, explains decisions, and receives feedback from humans or downstream outcomes. The system logs these episodes as decision traces. This period is exploratory: it allows the system to learn which categories, features, thresholds, and exception patterns matter.

During the *running period*, a local model is trained on the accumulated traces. When a new case arrives, the

local model predicts a decision, confidence score, and scope assessment. If confidence is high, the case is within the approved scope, and the risk level allows local action, the local component receives the right to act. If not, the decision is deferred to the LLM, a human reviewer, or a hybrid procedure.

The overall mechanism can be summarized as follows:

1. Identify a repeated decision point in an agentic workflow.
2. Define the candidate decision set and the minimum information required for that decision.
3. Let the LLM decide during a learning period while logging context, options, rationales, accepted outcomes, feedback, overrides, uncertainty, and version metadata.
4. Convert traces into structured features, embeddings, exemplars, graph records, or retrieval indexes.
5. Train a local classifier, ranker, retrieval model, approval model, or policy model.
6. Calibrate confidence and define the local component's approved scope.
7. Deploy the local component with thresholds, audit logs, and escalation paths.
8. Monitor override rate, deferral rate, calibration, distribution shift, downstream quality, and decision-share movement.
9. Reopen learning or retraining when the local component becomes stale or when new decision categories emerge.

This phase structure avoids two extremes. The system does not give permanent broad authority to the LLM merely because the LLM was useful at the beginning. It also does not prematurely replace the LLM before the workflow has accumulated enough local evidence.

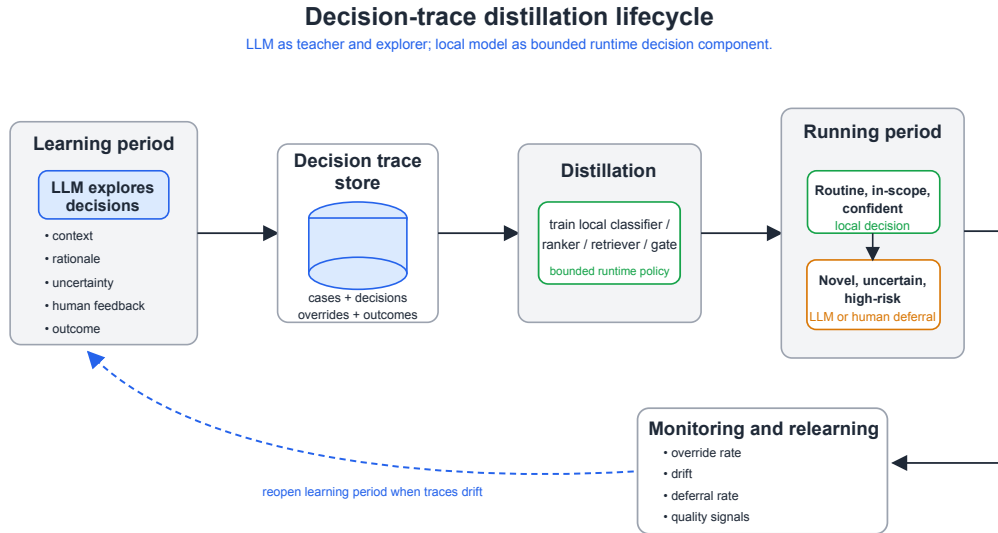


Figure 2: Decision-trace distillation lifecycle.

The lifecycle figure (Figure 2) shows the intended division of labor. The LLM acts as teacher and explorer during the learning period, while the local model acts as a bounded runtime component during the running period. Monitoring closes the loop: high override rates, drift, deferral spikes, or declining quality can reopen the learning period. This feedback path is important because localization should be reversible when the decision environment changes.

Five modes of local decision rights

The local model does not need to fully replace the LLM. Decision rights can be reduced gradually. We distinguish five modes:

Mode	Local model’s right	LLM’s role	Appropriate use
Retrieval support	Right to retrieve similar prior cases	Makes the final decision with precedents visible	Novel, ambiguous, or higher-risk tasks where precedent helps but should not decide
Recommendation	Right to propose one or more likely options	Accepts, revises, rejects, or explains disagreement	Repeated but moderately ambiguous decisions
Approval gate	Right to approve, reject, or flag an LLM proposal	Generates the candidate decision	Brand, policy, safety, or consistency checks
Direct decision	Right to decide above confidence and scope thresholds	Handles deferred cases	Routine, low-risk, finite-choice decisions
Voting member	Right to participate in an ensemble or arbitration process	Votes or supplies reasoning alongside other components	Decisions where diversity and disagreement are useful

This graded design is crucial. Reducing LLM decision rights does not always mean direct replacement. Sometimes the appropriate design is to make the LLM less unchecked by surrounding it with local precedents, deterministic review, or calibrated disagreement. In high-risk or weakly labeled domains, the local model may remain a retrieval or approval component for a long time. In routine low-risk domains, it may eventually make direct decisions for most cases.

The graded-rights figure (Figure 3) makes the governance implication explicit. As local components move from retrieval support to direct decision or voting, they gain more runtime authority and therefore require stronger calibration, audit, and scope control. The same technical artifact can therefore have different governance meanings depending on the right it receives. A classifier that only recommends an option is not equivalent to a classifier that may decide without LLM or human review.

Decision-right thresholding

The decision-right threshold determines when the local model is allowed to act. A simple version is:

$$\text{local_decide}(x) = \begin{cases} 1, & \text{if } p(\hat{y} \mid x) \geq \tau_d \text{ and } x \in S_d \text{ and } r_d(x) \leq \rho_d, \\ 0, & \text{otherwise.} \end{cases}$$

Here x is the case representation, $\hat{y}$ is the local model’s predicted decision, $p(\hat{y} \mid x)$ is estimated confidence,

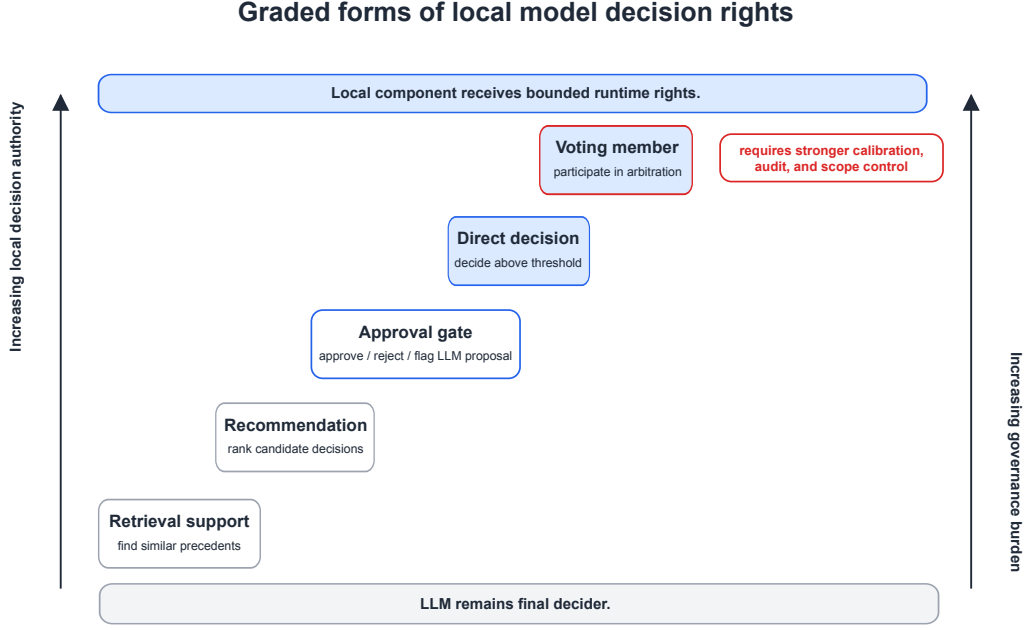


Figure 3: Graded forms of local model decision rights.

τ_d is the confidence threshold for decision point d , S_d is the approved scope of the local model at that decision point, and $r_d(x)$ is a risk score compared with an allowable risk bound ρ_d . Scope constraints prevent the local model from acting outside the distribution for which it was designed. Risk constraints prevent the local model from receiving direct authority even when it is confident but the consequences of error are too high.

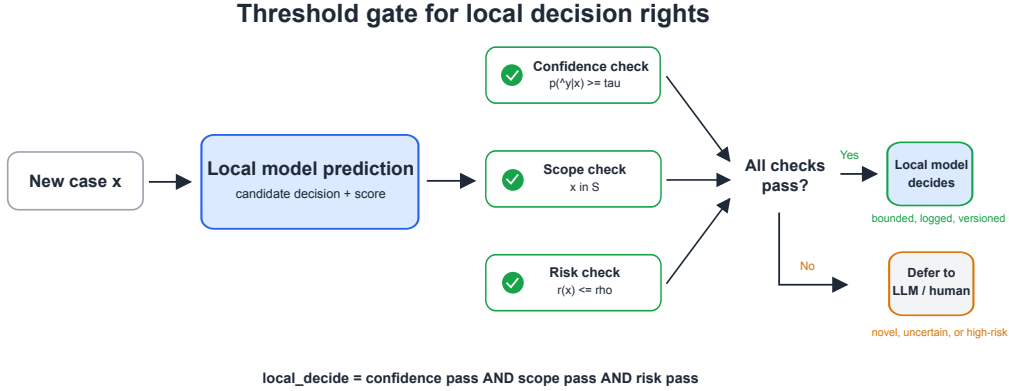


Figure 4: Threshold gate for local decision rights.

The threshold gate figure (Figure 4) translates the equation into an operational rule. A local decision is allowed only when confidence, scope, and risk checks all pass. If any check fails, the system defers. The design therefore treats deferral as a normal governance outcome, not as a failure of the local model.

The threshold makes decision-right allocation tunable. A higher threshold gives fewer direct rights to the local model and leaves more cases with the LLM or human reviewer. A lower threshold transfers more runtime authority to the local model but may increase override risk. The threshold can differ by decision type.

A cover-image theme may allow a lower threshold because the consequence of error is modest. Economic insight selection, credit decisions, safety interventions, or external actions should require stricter thresholds and possibly human approval.

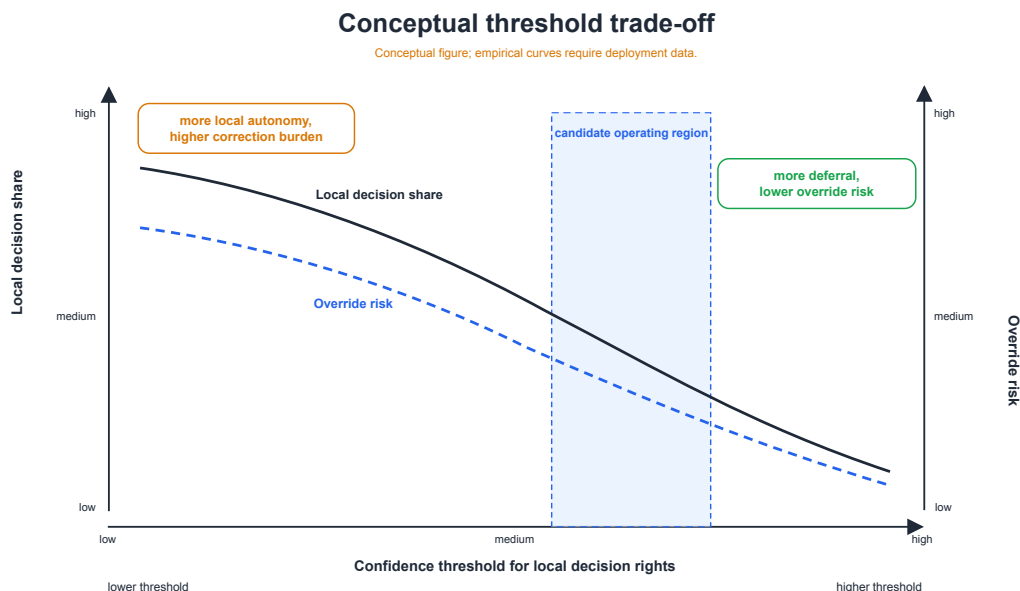


Figure 5: Conceptual threshold trade-off.

The threshold trade-off figure (Figure 5) is conceptual rather than empirical. It illustrates the expected relationship that must be estimated in deployment: as the confidence threshold rises, the local decision share should generally fall, and override risk should also fall. The candidate operating region is not the point that maximizes local autonomy. It is the region where the system reduces unnecessary LLM decision share while keeping override risk and deferral burden within the organization’s tolerance.

Relearning, reconsideration, and governance

A system that reduces LLM decision rights must still preserve adaptability. The local model can become stale when the user population, brand strategy, market environment, task definition, tool set, or model provider changes. Therefore, relearning is not an optional add-on. It is part of the design pattern.

The relearning module monitors deferral frequency, confidence decline, override rate, disagreement with the LLM, human correction patterns, downstream quality, and distribution shift. When these indicators cross a threshold, the system reopens the learning period for that decision point. This is a form of reconsideration: the system asks whether a right that had been localized should remain local, be retrained, be narrowed, or return to the LLM or human reviewer.

This governance loop should preserve traceability. Local decisions should be logged with the model version, feature representation, confidence, threshold, scope rule, retrieved precedents, and any later override. This supports system-level accountability, where the question is not merely why a model emitted a text string, but how a sequence of decisions produced an operational outcome (Zhu et al. 2026). It also supports targeted

guardrail placement because the system can identify which decision points propagate the most risk ([Ghosh et al. 2025](#)).

Use cases

The initial use cases are intentionally modest. The purpose is not to prove that all agentic decisions can be localized. It is to show how decision-trace distillation works for repeated decisions whose input context, candidate set, feedback signal, and risk level can be made explicit.

Article-structure selection

The first use case is selecting the structure of a Cognaptus blog article. In the initial workflow, an LLM reads the topic, target audience, source material, intended message, desired tone, length constraint, and publishing context. It then selects an article structure, such as explanatory essay, problem–solution, trend analysis, case-based argument, data commentary, opinion note, or executive briefing.

During the learning period, each decision is recorded with topic features, audience, source density, desired argument style, tone, final human-selected structure, revision burden, and publication outcome. After enough examples accumulate, a local classifier can predict the preferred structure for familiar cases. At runtime, the classifier may directly choose the structure above threshold. When confidence is lower, it can provide the top candidate structures to the LLM together with similar precedents.

This use case is suitable because the decision is repeated, the output classes are finite, and human feedback is available. It also has measurable downstream signals: whether the selected structure leads to fewer revisions, faster acceptance, or better fit with the intended message. The main risk is over-localization. If the local model learns a narrow house style too strongly, it may suppress useful novelty. The threshold and relearning module should therefore monitor topic novelty and editor overrides.

Cover-image theme selection

The second use case is selecting a cover-image theme for a Cognaptus blog article. The LLM initially chooses among themes such as abstract concept, human collaboration, data visualization, market metaphor, technology interface, industry scene, or minimalist symbolic design. This choice depends on article title, topic, audience, intended emotional tone, source material, and brand memory.

A local model can learn from previous LLM and human decisions. For example, articles about organizational strategy may often use abstract or metaphorical imagery, while data-heavy macroeconomic notes may use chart-based or market-themed imagery. The local model can then select or recommend a theme when the article matches previous patterns. For ambiguous or novel topics, the decision is deferred to the LLM or to a human editor.

This use case separates creative generation from decision control. The local model does not need to generate the image. It only selects the theme, approves whether a proposed theme is consistent with brand memory, or retrieves similar prior design decisions. A generative image model can still create candidate images, while the local component governs part of the decision process.

Economic-data insight selection

The third use case is selecting the main insight from economic data. Compared with the first two use cases, this decision is more difficult and consequential. The input may include time series, metadata, market expectations, revisions, calendar context, historical regimes, related indicators, and domain-specific interpretation. The output may be an insight frame, such as trend continuation, structural break, anomaly, policy implication, sector divergence, cyclical turning point, or risk alert.

For this use case, the local model should not receive full direct decision rights at the beginning. A better first role is retrieval support, recommendation, or voting. It can retrieve similar historical cases, classify the likely insight type, identify whether the current pattern resembles known episodes, and flag when the observation is outside previous experience. The LLM can then use this local precedent as a constraint or reference rather than deciding from scratch.

This use case illustrates the difference between reducing and eliminating LLM decision rights. The LLM may remain valuable for synthesizing macroeconomic context, explaining trade-offs, and generating narrative implications. The local model reduces unchecked authority by anchoring the LLM to precedent, scope, and historical patterns. Over time, if the decision set becomes sufficiently stable and feedback signals become reliable, the system may allow direct local decisions for narrow low-risk subcases while preserving LLM or human review for novel or high-impact interpretations.

Evaluation plan

The empirical evaluation should ask whether decision-trace distillation can reduce runtime LLM decision share while preserving or improving accepted decision quality, stability, interpretability, and governance. This is different from asking whether the local model always outperforms the LLM. A successful hybrid system may preserve most accepted decisions while shifting routine decisions away from the LLM and escalating uncertain cases.

The evaluation should be framed as a comparative system study rather than as a broad causal claim about LLMs or local models. The treatment of interest is a workflow design: whether a repeated decision point is handled by an LLM-only process, a local-only process, a thresholded hybrid process, or a conservative support process. The strongest initial evidence will come from fixed-task comparisons in which each variant receives the same input cases, candidate options, logging schema, and outcome labels.

Research questions

The first evaluation round can be organized around four research questions:

1. **Decision-share reduction:** How much of the runtime decision share can be shifted from the LLM to the local model under different thresholds?
2. **Quality preservation:** Does the hybrid system match final human-accepted or production-accepted decisions at a rate comparable to the LLM-only baseline?
3. **Governability:** Are repeated decisions more stable, traceable, and easier to inspect when handled by local components?

4. **Risk-sensitive deferral:** Does the system defer novel, uncertain, or high-risk cases rather than over-confidently localizing them?

These questions align with the broader move in agent evaluation away from final-output scoring toward trajectory, process, artifact, safety, and long-horizon assessment (Yehudai et al. 2025; Zhuge et al. 2024). They also reflect the autonomy literature’s emphasis on treating autonomy as a design variable rather than a direct consequence of model capability (Feng, McDonald, and Zhang 2025).

Data construction

The evaluation dataset should be built at the decision-point level rather than at the whole-workflow level. Each row should represent one decision opportunity, such as selecting an article structure, choosing a cover-image theme, or assigning an economic insight frame. The minimum record should include the input context, candidate option set, LLM-selected option, human-accepted or production-accepted option, override reason if any, downstream quality signal, uncertainty signal, and risk or scope tag.

For article-structure and cover-image use cases, retrospective logs can provide the first dataset. Earlier article topics, LLM recommendations, editor-selected structures, image themes, revision outcomes, and publication decisions can be converted into supervised examples. A chronological split is preferable to a random split because the real deployment problem is whether earlier traces can support later decisions under changing topics and editorial preferences.

For the economic-data insight use case, retrospective evaluation should begin with support roles. Human experts can assess whether retrieved precedents are relevant, whether suggested insight frames are plausible, and whether the system appropriately flags novel or uncertain cases. Only after this support role is validated should the local model receive narrow direct decision rights.

System variants

The evaluation should compare at least four variants:

Variant	Description	Purpose
LLM-only	The LLM makes the decision at each decision point.	Baseline for quality, cost, latency, and stability.
Local-only	The local model decides whenever it has a prediction.	Stress test for over-localization risk.
Hybrid threshold	The local model decides only above threshold and defers other cases.	Main proposed design.
Hybrid support	The local model retrieves, recommends, approves, or votes, but the LLM or human remains final decider.	Conservative design for higher-risk tasks.

The local-only variant is not necessarily a recommended deployment pattern. It is useful because it shows what happens when too many rights are shifted away from the LLM. The hybrid variants are more important for practical design because they reveal the trade-off between local decision share, quality, and override risk.

Experimental protocol

A practical first protocol has six steps. First, choose one decision point and freeze its candidate option set for the initial evaluation period. Second, construct a training set from earlier accepted traces and a validation set for threshold selection. Third, train one or more local components, such as a classifier for finite options, a ranker for top- k recommendations, or a retrieval model for precedent search. Fourth, calibrate confidence on the validation set and define scope rules, including novelty indicators and excluded case types. Fifth, evaluate all system variants on a chronologically later test set. Sixth, repeat the evaluation across a threshold sweep so that the trade-off among local decision share, deferral rate, accepted-decision agreement, and override risk can be observed.

This protocol should avoid three common overstatements. First, matching the LLM’s raw decision is not the main target when human-accepted or outcome-validated decisions are available. Second, high local accuracy is insufficient if the local model acts outside its approved scope. Third, low deferral is not automatically good; for high-risk or novel cases, deferral is the desired behavior.

Metrics

The following metrics should be reported:

Metric	Meaning
Agreement with accepted decision	Whether the system matches the final human-accepted or production-accepted decision.
Downstream quality	Whether the resulting article, image theme, or insight requires fewer revisions or produces better task outcomes.
LLM decision share	Percentage of cases where the LLM makes the final decision.
Local decision share	Percentage of cases where the local model makes the final decision.
Deferral rate	Percentage of cases escalated to the LLM or human reviewer.
Override rate	Percentage of local decisions later corrected by the LLM, human, or downstream evaluation.
Stability	Consistency of decisions across repeated runs on the same input.
Calibration	Relationship between predicted confidence and observed correctness.

Metric	Meaning
Scope violation rate	Frequency with which the local model acts outside its approved domain.
Interpretability	Ability to inspect features, precedents, rules, thresholds, and model versions behind a decision.
Cost and latency	Reduction in model calls, response time, and operational expense.

The central expected output is a threshold curve. As the confidence threshold decreases, the local model receives more direct decision rights, cost and latency may fall, and stability may improve. At the same time, override risk may rise. The desirable threshold is not universal. It depends on task risk, user tolerance, availability of human review, and the cost of mistakes at scale.

Prospective monitoring and relearning criteria

Retrospective evaluation is only the first step. A prospective deployment should monitor the same decision point after the local component receives bounded rights. The monitoring dashboard should report local decision share, deferral rate, override rate, calibration drift, category-level error concentration, and changes in the distribution of topics or input features. These metrics should be tracked by decision type rather than only in aggregate, because a stable average can hide deterioration in a newly important category.

Relearning should be triggered when predefined indicators cross governance thresholds. Examples include a sustained increase in human overrides, a rise in high-confidence errors, a decline in calibration, a spike in out-of-scope cases, or the appearance of new candidate options. When a trigger fires, the system should either reopen the learning period, narrow the local model’s scope, raise the decision threshold, or return the decision right to the LLM or human reviewer until updated traces support relocalization.

Contributions

This paper makes three contributions.

First, it reframes model distillation as a decision-rights allocation mechanism. Traditional distillation and model compression transfer predictive behavior from a larger teacher into a smaller student ([Buciluă, Caruana, and Niculescu-Mizil 2006](#); [Hinton, Vinyals, and Dean 2015](#)). Decision-trace distillation transfers a narrower object: the authority to make a repeated workflow decision under explicit scope, confidence, and risk constraints.

Second, it specifies a graded design pattern for reducing LLM runtime authority. The local model can retrieve precedents, recommend options, approve LLM proposals, decide directly above threshold, or participate in voting. This graded view prevents the false choice between full LLM autonomy and full deterministic replacement. It also connects agentic AI governance to the practical design of thresholds, deferral, audit logs, and retraining triggers.

Third, it links local memory, knowledge compilation, and system-level accountability. Agent memory is often framed as a way to improve future LLM outputs. This paper adds another interpretation: when memory is converted into a classifier, ranker, approval gate, or retrieval model, it becomes a local decision component. The result is a more inspectable agentic system in which familiar decisions can be governed locally and unfamiliar decisions can remain adaptive.

Limitations

This paper is an initial design proposal and does not claim that local models should replace LLMs in all agentic decisions. Some decisions are rare, open-ended, creative, socially sensitive, or highly context-dependent. In those cases, the LLM may remain the most appropriate semantic reasoner, and the local model should only retrieve precedents, provide checks, or assist human review.

The proposal also depends on the quality of logged decisions. If the learning period records poor LLM decisions, biased outputs, weak rationales, or unrepresentative human feedback, the local model may reproduce those problems. The most important training target is therefore not the raw LLM decision but the final accepted or outcome-validated decision. Human corrections, downstream performance, and override reasons should be treated as first-class data.

Another limitation is distribution shift. A local model trained on past decisions may become outdated when the brand strategy, audience, market environment, tool set, model provider, or task definition changes. Re-learning and reconsideration are therefore essential. A system should be able to narrow, suspend, retrain, or revoke local decision rights when performance indicators deteriorate.

A further limitation is measurement. Effective decision rights may differ from formal rights. A human may formally approve an output but lack enough information to challenge it. A local model may formally only recommend an option but effectively anchor the LLM’s later choice. An LLM may formally only generate rationales but shape the option set in ways that determine the final decision. Future empirical work should estimate effective rights from logs, interface behavior, routing records, memory writes, approval patterns, and failure cases.

Finally, the pattern may create new governance responsibilities. Local models can appear more stable and transparent than LLMs, but they still require calibration, monitoring, documentation, and scope control. A poorly calibrated local model can silently receive too much authority. Decision-trace distillation should therefore be deployed as a governed workflow pattern, not merely as a technical optimization.

Conclusion

Agentic AI systems gain adaptability by giving LLMs broad decision rights. However, broad LLM decision rights also create practical problems of stability, reproducibility, transparency, and accountability. This paper proposed decision-trace distillation as a design pattern for reducing unnecessary LLM runtime authority while preserving LLM adaptability for novel, uncertain, and high-risk cases.

The mechanism uses the LLM heavily during a learning period, records decision traces, and then trains local

deterministic or semi-deterministic components for recurrent decisions. The local component may retrieve, recommend, approve, decide, or vote depending on confidence, scope, and risk. The LLM becomes a teacher, fallback decision maker, and explorer of novel cases rather than the permanent holder of all workflow authority.

The broader research agenda is to make decision authority an explicit and configurable property of agentic systems. Instead of implicitly giving the most general model the most authority, designers should ask which party should hold which right at each decision point. Decision-trace distillation offers one practical answer: let LLMs help discover local regularities, then shift repeated decisions into more bounded, auditable, and governable components when the evidence supports doing so.

References

- Buciluă, Cristian, Rich Caruana, and Alexandru Niculescu-Mizil. 2006. “Model Compression.” *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 535–41. <https://doi.org/10.1145/1150402.1150464>.
- Chen, Lingjiao, Matei Zaharia, and James Zou. 2023. “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.” <https://arxiv.org/abs/2305.05176v1>.
- Cheng, Edward, and Jeshua Cheng. 2026. “A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows.” <https://arxiv.org/abs/2604.23049v1>.
- Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. “Levels of Autonomy for AI Agents.” <https://arxiv.org/abs/2506.12469v2>.
- Fiorini, Sandro Rama, Leonardo G. Azevedo, Raphael M. Thiago, Valesca M. de Sousa, Anton B. Labate, and Viviane Torres da Silva. 2025. “Episodic Memory in Agentic Frameworks: Suggesting Next Tasks.” <https://arxiv.org/abs/2511.17775v1>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. “A Safety and Security Framework for Real-World Agentic Systems.” <https://arxiv.org/abs/2511.21990v1>.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. “Distilling the Knowledge in a Neural Network.” <https://arxiv.org/abs/1503.02531v1>.
- Lai, Guannan, and Han-Jia Ye. 2026. “When Routing Collapses: On the Degenerate Convergence of LLM Routers.” <https://arxiv.org/abs/2602.03478v1>.
- Lamo Castrillo, Victor de, Habtom Kahsay Gidey, Alexander Lenz, and Alois Knoll. 2025. “Fundamentals of Building Autonomous LLM Agents.” <https://arxiv.org/abs/2510.09244v1>.
- Lu, Ruofan, Yichen Li, and Yintong Huo. 2025. “Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks.” <https://arxiv.org/abs/2508.13143v1>.
- Ong, Isaac, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. “RouteLLM: Learning to Route LLMs with Preference Data.” <https://arxiv.org/abs/2406.18665v4>.
- Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. “AI Agents Vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges.” <https://doi.org/10.1016/j.inffus.2025.103599>.
- Singh, Yash Raj. 2025. “Graph-Memoized Reasoning: Foundations Structured Workflow Reuse in Intelli-

- gent Systems.” <https://arxiv.org/abs/2511.15715v1>.
- Tang, Xiangru, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, et al. 2025. “Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving.” <https://arxiv.org/abs/2507.06229v5>.
- Tsiourvas, Asterios, Wei Sun, and Georgia Perakis. 2025. “Causal LLM Routing: End-to-End Regret Minimization from Observational Data.” <https://arxiv.org/abs/2505.16037v2>.
- Uulu, Choro Ulan, Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, and Helena Holmström Olsson. 2026. “How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework.” <https://arxiv.org/abs/2601.15153v1>.
- Varangot-Reille, Clovis, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. 2025. “Doing More with Less: A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems.” <https://arxiv.org/abs/2502.00409v3>.
- Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. 2023. “A Survey on Large Language Model Based Autonomous Agents.” <https://doi.org/10.1007/s11704-024-40231-1>.
- Wang, Wenshuo. 2026. “LLMs Should Not yet Be Credited with Decision Explanation.” <https://arxiv.org/abs/2605.01164v1>.
- Wissuchek, Christopher, and Patrick Zschech. 2025. “Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework.” <https://arxiv.org/abs/2508.00844v1>.
- Wray, Robert E., James R. Kirk, and John E. Laird. 2025. “Applying Cognitive Design Patterns to General LLM Agents.” <https://arxiv.org/abs/2505.07087v2>.
- Yehudai, Asaf, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. “Survey on Evaluation of LLM-Based Agents.” <https://arxiv.org/abs/2503.16416v1>.
- Zhang, Guangwei. 2025. “Knowledge Protocol Engineering: A New Paradigm for AI in Domain-Specific Knowledge Work.” <https://arxiv.org/abs/2507.02760v1>.
- Zhou, Oliver, and Lily Li. 2026. “Decision-Rights Allocation: A Governance-Centric Perspective on LLM-Enabled Agentic Systems.” Quantum Capital AI Ltd. https://oliverlds.github.io/files/decision_rights_allocation_concept.pdf.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability.” <https://arxiv.org/abs/2601.17168v1>.
- Zhuge, Mingchen, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, et al. 2024. “Agent-as-a-Judge: Evaluate Agents with Agents.” <https://arxiv.org/abs/2410.10934v2>.